Can AI Build Systems? An Exploratory Study on Generating Software Architecture with LLMs

Marios Fokaefs
York University
Toronto, Canada
fokaefs@yorku.ca

Hashim Khan
York University
Toronto, Canada
hashimahmedkhan2002@gmail.com

Borna Ahmadzadeh
York University
Toronto, Canada
bornaahz@my.yorku.ca

***Abstract*—Software architecture is instrumental in shaping the quality, maintainability, and integrity of a software system, providing a blueprint that ensures that all components function cohesively. Despite being introduced only recently, Large Language Models (LLMs) have shown great potential in automating low-level software engineering tasks, such as coding and test generation. However, the application of LLMs on high level cognitive tasks, such as architecture and design, has not been explored as much. To that end, this study evaluates the capability of LLMs—specifically Llama 4 Llama, Gemini 2.0 Gemini, and OpenAI o3 in generating software architecture given unstructured textual requirements in natural language. We employ prompt engineering techniques, including zero-shot prompting, in-context learning and chain-of-thought prompting to assess each model's responsiveness and accuracy. Furthermore, we evaluate iterative refinement, an approach where an LLM revises the output of another for enhanced quality, akin to a human editing a preliminary draft for improvements. Our results indicate that state of the art models like OpenAI o3 and Gemini 2.0 Gemini perform exceptionally well in identifying all the domain concepts of the system and how they interact, especially in conjunction with iterative refinement, while Llama' Llama 4's performance was subpar. In terms of generating an architecture for the system, again models from OpenAI and Gemini are able to come close to the benchmark architecture by using the correct architectural styles especially with Chain-of-Thought prompting, however Llama's model do not infer the correct styles.**

***Index Terms*—software architecture, large language models, UML, software design, software requirements, microservice architecture**

I. Introduction

Software architecture serves as the foundational structure of a software system, defining its components, their interactions, and the guiding principles for its design and evolution. As software systems grow increasingly complex, the role of software architecture extends beyond algorithms and data structures to include considerations such as system organization, control structures, data access protocols, scalability, and performance optimization [1]. Architectural decisions shape the maintainability, extensibility, and efficiency of software systems, ensuring they meet both functional and non-functional requirements.

Historically, software architecture has been formalized through various design models and industrial approaches, incorporating modularization, layered designs, client-server paradigms, and domain-specific frameworks [2]. These methodologies enable systematic development while maintaining intellectual control over system complexity. However, despite advances in architectural modeling, traditional methods remain reliant on manual design expertise, requiring significant time and effort to ensure architectural consistency, adaptability, and performance. Moreover, mistakes during the design phase of the architecture can show up later, imposing high maintenance costs. Therefore, we require means to support software architects in their architectural design decisions and evaluation to reduce the time required and make the resulting architectures more reproducible through semi-automation.

The advent of Large Language Models (LLMs) has introduced a transformative shift in software engineering, enabling automation in various technical tasks. LLMs, such as OpenAI's GPT models [3], leverage deep learning techniques to process vast amounts of textual and code-based data, generating human-like responses, summarizing information, and even writing and debugging software code. The application of LLMs in software engineering spans multiple areas, including code generation, software testing, and requirement analysis. However, unlike code generation, which can be considered a highly structured and well defined process, architecting software systems is a highly cognitive and creative task that is only guided by good practices and principles. As such it is generally unknown, and largely unexplored, how well modern generative AI methods can fair in such a task.

To explore the effectiveness of LLMs in software architecture generation, this study focuses on a central research question:

- **RQ:** To what extent can LLMs automatically extract requirements, infer appropriate domain concepts and their relationships, and produce a high-level UML component architecture for a system—adhering to established architectural styles—using only a natural-language description of the system ?

To address this question, we conducted an exploratory study that combines three prompting techniques—zero-shot, in-context, and chain-of-thought—with three state-of-the-art LLMs: OpenAI o3[1], Gemini 2.0 Flash[2], and Llama 4 Maverick[3]. For each model-prompt pairing we fed only the textual description of two real open-source projects and asked the

[1]https://openai.com/index/introducing-o3-and-o4-mini/
[2]https://deepmind.google/models/gemini/Gemini/
[3]https://ai.meta.com/blog/llama-4-multimodal-intelligence/

model to output a high level UML component diagram in the PlantUML toolsuite[4]. We then compared every generated diagram to the project's original architecture, evaluating how well it (i) captures the stated requirements, (ii) identifies the right domain entities and relationships, and (iii) follows—or thoughtfully improves upon—the architectural styles chosen by the human developers. This research aims to provide insights into how AI-driven approaches can reshape software architecture practices, accelerate development cycles, and enhance the accessibility of high-quality architectural outputs.

Our experimental scripts and results are made available to enable the independent verification and replication of the results presented in this study[5].

The rest of the paper is organized as follows. Section II outlines research relevant to the present work. Section III outlines the overall methodology of our study, while Section IV presents the experiment details. Section V-D outlines the results of the experiment performed in this study and finally, VI presents related discussion and limitations of.

II. Related Work

The generation of software architecture from requirements has been a long-standing challenge in software engineering. Souza et al. [4] conducted a systematic mapping study on deriving architectural models from requirements specifications, highlighting the reliance on architects' tacit knowledge and the lack of support for inexperienced practitioner. Hofmeister et al. [2] introduced a general model for software architecture design derived from five industrial approaches, consolidating commonalities among methodologies like Attribute-Driven Design (ADD), Rational Unified Process (RUP), and Siemens' 4 Views, emphasizing iterative cycles of architectural analysis, synthesis, and evaluation. Our work is using this general model as the basis for guiding the LLMs.

With the rise of Large Language models (LLMs), researchers have explored their potential to transform software engineering processes. Pothukuchi et al. [5] examined the impact of generative AI on the software development lifecycle (SDLC), demonstrating how LLMs accelerate planning, design, coding, testing, and maintenance phases while reducing workforce requirements. Similarly, Zheng et al. [6] provided a comprehensive review of LLM applications in software engineering, categorizing tasks like code generation, vulnerability detection, and software documentation, while highlighting LLMs' strengths in syntax-based tasks and limitations in semantic comprehension.

For modeling and design artifacts, recent work has tested LLMs on creating UML models from natural language. Ferrari et al. [7] generate UML sequence diagrams and report readable, standard-compliant outputs that nonetheless exhibit gaps when the text is ambiguous. Cámara et al. [8] evaluate ChatGPT on UML class diagrams (with OCL), noting variability across runs and semantic inconsistencies despite acceptable syntax. In contrast, our study targets UML component diagrams to recover high-level architecture (components and connectors) rather than interaction- or class-level structure, enabling us to assess style conformance and global similarity—via edit distance—alongside element-level precision/recall.

Related to design, architecture, and document generation, Krishna et al. [9] studied whether LLMs can be leveraged to create Software Requirements Specifications (SRS). Their experiments demonstrated that GPT-4 could match entry-level engineers in generating comprehensive SRS documents, although this depended on carefully crafted prompts and domain-specific context. Dhar et al. [10] investigated whether LLMs could produce Architectural Design Decisions (ADDs). They found that while LLMs can propose plausible decisions given structured context, the models sometimes lack comprehensive rationales or introduce inconsistencies in multi step reasoning. Both efforts highlight the need for significant human intervention to refine outputs, and raise concerns about consistency and reliability, particularly for complex or domain-specific architectural designs.

Focusing on software architecture generation via human-bot collaboration, Ahmad et al. [11] proposed an iterative approach using ChatGPT to guide architects through requirements analysis, architectural model synthesis, and subsequent evaluation. Their method involves multiple conversation loops between the architect and ChatGPT, with the human providing incremental feedback. While this interactive guidance proved effective, it demands substantial domain knowledge from the user to steer the discussion, and the final architecture still relies heavily on an architect's manual validation of the LLM's outputs. Similarly, Eisenreich et al. [12] presented a semi-automatic method to translate requirements into architecture candidates, integrating domain-driven design techniques with LLM-generated architecture models. They emphasized iterative refinement and trade-off analyses, employing LLMs to suggest multiple architecture options aligned with both functional and non-functional requirements, with the final selection made manually. However, their research has thus far explored only the use of LLMs in generating domain models. The task of generating architectural candidates from these models and evaluating them using LLMs remains unaddressed.

Our approach builds on these findings by integrating a best-practice architectural process with iterative LLM interactions, targeting both architecture generation and design documentation. Compared to the incremental conversation loops of the work of Ahmad et al. [11], our approach organizes LLM calls into clearly defined phases for requirements analysis, synthesis, and evaluation, aiming to reduce the reliance on continuous architect-model dialogues. Unlike the work by Eisenreich et al. [12], which focuses mainly on domain-driven design abstractions, we extend domain modeling to full architecture generation, combined with an automatic or semi-automatic evaluation stage to help architects select from among multiple architecture candidates. Although we uphold the necessity for human oversight to validate LLM outputs,

[4]https://plantuml.com/

[5]https://anonymous.4open.science/r/Can-AI-Build-Systems-An-Exploratory-Study/README.md

our methodology aims to provide more structured, end-to- end support for architects, ranging from early analysis to design-level detailing.

III. Methodology

We applied various prompt engineering techniques to evaluate the performance of different Large Language Models (LLMs) in generating architecture of software systems given their textual descriptions in natural langauge. Specifically, we benchmarked three prominent LLMs under each of these prompting strategies: **OpenAI o3**, the latest reasoning-tuned successor to the GPT-4 family; **Gemini 2.0 Gemini**, Google's latency-optimised variant of Gemini designed for rapid iterative calls; and **Llama 4 Llama**, Meta's fourth-generation open-weight model fine-tuned for software-engineering tasks. These models are selected based on popularity and their wide applicability on software development tasks. We evaluate two complementary workflows for turning a natural-language system description into a PlantUML component diagram. We decided to use UML, particularly PlantUML for modeling due to a number of factors, such as available documentation, ease of use, tool support, and wide-scale adoption as a language to represent software systems [13]. This ubiquity also means contemporary LLMs have substantial prior exposure to UML outputs, which in practice reduces syntactic errors in generated diagrams.

Single-Pass Generation: We first apply "Zero-shot" prompting [14], "In-context" learning [15] and "Chain-of-thought" prompting [16], which are well-established prompt-engineering strategies, requiring the model to infer a diagram without anyother external help, such as a dedicated knowledge base. **In-context learning** embeds relevant examples directly inside the prompt so that the model can align its output with previously observed patterns. **Chain-of-though prompting** instructs the model to generate step-by-step reasoning thereby fostering more transparent and decomposed generation. It has proven effective for complex queries that require multi-step solutions. All nine model-prompt combinations received identical inputs and produced PlantUML component diagrams that were subsequently parsed for evaluation in the following subsections.

Two-Pass Iterative Refinement: Inspired by how human architects sketch a draft and then polish it, we evaluate a similar iterative refinement approach that has been previously explored in areas such as natural language translation [17], mathematical reasoning [18], and coding [19]. Concretely, we adopt a two-pass strategy consisting of two LLMs chained together. The first model is responsible for generating a preliminary output, which is subsequently passed to the second model to have its issues diagnosed and appropriately fixed to enhance overall quality and ensure correctness. In our domain, the first model may establish the overall structure of the software architecture, such as crucial components and connections, while the second one analyzes this initial response, identifying and rectifying missing or redundant interfaces, anti-patterns, confused module responsibilities, or other problems that could degrade the architecture. Because each of the nine model–prompt settings can serve in either the "drafter" or "refiner" role, we evaluate **all 81 ordered pairs (9 × 9)**.

Together, the single-pass and iterative experiments let us assess: (i) what levels of accuracy, compared to the intended architecture, the best model–prompt combination can reach on its own, and (ii) how much additional gain is unlocked when a second model is invited to critique and correct the initial design. This comparative analysis identifies methods that improve performance and assesses the strengths and limitations of current generative AI technology in generating software architecture from textual descriptions. The findings provide insights into the feasibility of employing generative models as co-pilots for a faster more reliable process of archtitecture generation.

IV. Experiment

A. Data and Baseline

We began by mining public GitHub repositories with one or more of the following keywords: **"microservices", "event-driven", "architecture"**. The focus on microservices and event-driven architectures was due to the fact repositories that follow these architectures tend to document the components or services of their system in detail, which provides enough information for the LLM. Using GitHub Search, we queried repositories with such keywords. We filtered these repositories down to **25 repositories** whose README files included at least brief description of the system and contained at least one diagram illustrating the system's overall structure. This first pass eliminated more implementation specific repositories, leaving projects with enough narrative context to ground an architectural discussion.

From that shortlist we selected two systems whose system description captured every salient feature or business capability, the supplied diagram depicts a high-level architecture and the domain logic is non-trivial. Choosing a pair of reasonably complex yet fully documented systems allows us to test whether LLMs can infer both breadth (all major components) and depth (domain-specific responsibilities).

For each chosen repository we extracted textual description by stripping out implementation specific information such as language, framework, or container orchestration script and retained only sentences that describe functional and non functional requirements, domain entities, and inter-service interactions. Separately, we recreated the project's original architecture diagram as a PlantUML component diagram, normalizing notation so that every baseline as well generated diagram shares the same semantic granularity (components and directed connectors). This baseline acts as the reference against which every LLM-generated diagram is compared.

Because the baseline reflects the intended architecture as documented in the repository, it lets us gauge how effectively each model: (i) understands the developer's intent embodied in the textual description, (ii) captures all domain-critical components and their relationships, and (iii) preserves—or

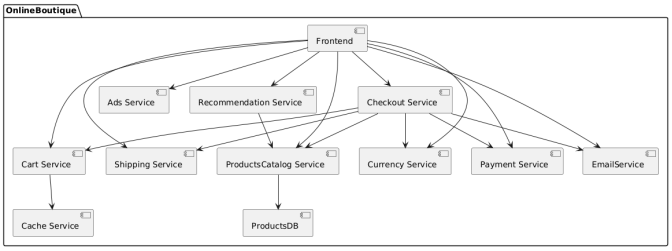


Fig. 1: Online Boutique Architecture

thoughtfully improves—the architectural style originally advocated by the project (e.g., microservice boundaries, event buses, Command Query Responsibility Segregation (CQRS)). Consequently, discrepancies between a generated diagram and the baseline speak directly to an LLM's potential (or limitation) as an architect.

The two repositories that satisfied all selection criteria and therefore serve as our benchmarks are described below.

The first, **Online Boutique**, comes from Google Cloud's *microservices-demo* project [6] It was released to demonstrate how an enterprise workload can be modernized on Google Cloud infrastructure (GKE, Cloud Service Mesh, gRPC, Cloud Operations). Functionally, the application is a fully featured online store: customers browse a product catalogue, manage a shopping cart, and complete checkout with multi-currency pricing, multiple payment methods, shipping estimates, and confirmation e-mails, while a responsive front-end serves personalized recommendations and contextual adverts. The reference architecture follows a textbook microservices style in which each domain concept is isolated in its own service. There is no API-gateway and services talk to one another directly. The architecture diagram extracted from the repository and converted to PlantUML component diagram is shown in Figure 1.

The second benchmark, **Meetings System**, is hosted in K. Grzybek's repository. [7]. The code base was created to showcase best practices for building a modular application that applies Domain-Driven Design (DDD). The system allows users to register as Members or Administrators, propose interest-driven meeting groups, and—once a proposal is approved—schedule fee-based meetings with capacity limits and guest allowances; a payment component ensures that only groups with valid subscriptions can continue creating events. Architecturally, the application exposes a single entry point for all external API requests similar to an API Gateway with an Event Driven Architecture relying on an internal event bus for modules to publish and consume domain events asynchronously, achieving loose coupling inside one deployable unit. The architecture diagram extracted from the repository and converted to PlantUML component diagram is shown in Figure 2.

[6]https://github.com/GoogleCloudPlatform/microservices-demo
[7]https://github.com/kgrzybek/modular-monolith-with-ddd

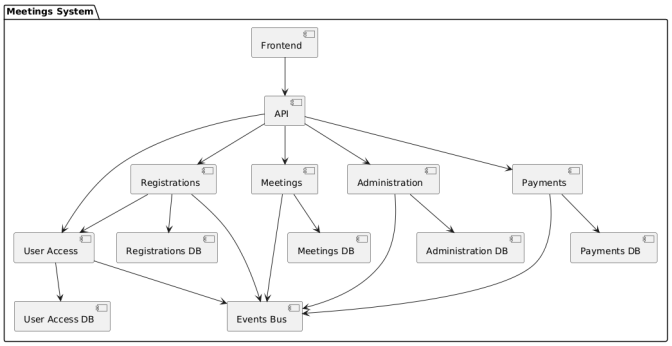


Fig. 2: Meetings System Architecture

B. Metrics

To judge how each model fared in generating an architecture compared with the benchmark, we adopted the same mixed-evaluation strategy for *both* workflows—the single-pass runs and the two-pass iterative refinements—combining element-level counts with a holistic structural distance.

First, every generated PlantUML component diagram was inspected against the baseline to tally **true positives (TP)**, **false positives (FP)** and **false negatives (FN)** for both components and connections. An element counted as a TP when it appeared in the generated diagram *and* the baseline, signalling that the model correctly identified a domain concept or relationship. Elements present only in the model's output were logged as FPs, revealing superfluous components or links, while elements missing from the output but present in the baseline were logged as FNs, highlighting requirements the model overlooked. Here to cater for the problem of models giving similar components slightly different names, we manually inspected the names of components and if the component had a similar name to the benchmark component we counted it as similar. For example, cases like 'Order Service' and 'Order' components were considered the same. Counting TP, FP and FN for components and for connections lets us compute precision/recall while also revealing *why* a diagram diverges: extra services inflate FP indicating the model may have hallucinated, whereas overlooked domain entities inflate FN failing to provide a complete overview for the architecture. The final precision, recall, and F1-score for components and connections combined correspond to the effectiveness of the model, while the sum of FP and FN show the limitations of the model.

Second, to capture overall similarity we computed the **tree-edit distance** between each pair of diagrams. We converted both baseline and generated PlantUML files to XMI, parsed each into an ordered, labelled tree, and then applied a Zhang–Shasha style algorithm [20] that returns the minimum sequence of insert, delete, and relabel operations required to transform one tree into the other. This metric returns the minimum number of edit operations needed to transform one diagram into the other, giving a single quantitative score that accounts for both missing and misplaced elements as well as

Zero-Shot Prompting:

I am providing the description of a system we want to design. Create a high-level architecture for the system. Create a PlantUML component diagram to model the architecture.

In-Context Learning:

I am providing the description of a system we want to design. Create a high-level architecture for the system. Create a PlantUML component diagram to model the architecture.

Context: You are a software architect experienced in designing and implementing software systems in different domains given their descriptions. The attached text gives the description and core functionality of a software system. Generate an architecture for this system that: 1. Adheres to architecture styles and software design principles 2. Keep the architecture high level, excluding any implementation and technology specific details.

Chain-of-thought Prompting

I am providing the description of a system we want to design. Create a high-level architecture for the system. Create a PlantUML component diagram to model the architecture by following these steps:

1. Consider the functional and non-functional requirements of the system.

2. Consider some main use cases of the system.

3. Use your domain knowledge about the system and come up with a domain model.

4. Using the domain modal, generate the high-level architecture of the system, leaving out any implementation and technology specific details.

Fig. 3: Sample prompts used to query the LLMs.

larger structural rearrangements.

C. Experiment Procedure

To automate experimentation we built a lightweight automation tool in Python using `LangChain`[8] as the orchestration layer. The tool exposes a simple UI in which the researcher pastes the system description, chooses the target LLM from a drop-down list, and selects one of the three prompt strategies (zero-shot, in-context, or chain-of-thought). Under the hood it uses LangChain's OpenAI API for o3, the GoogleGenAI API for Gemini, and Groq API for Llama, then processes the output from the LLM to extract PlantUML diagram code and create PlantUML component diagram files. This end-to-end pipeline guarantees identical pre-processing and post-processing across models.

Generation hyper-parameters were **standardised** to minimize variance. We set the temperature (t) to 0.5 to produce deterministic outputs but still allowing limited creativity and configured top˙p to 0.8 to control the diversity of text generation. The maximum output length was capped at 2,048 tokens

[8]https://www.langchain.com/

TABLE I: Performance of the models expressed in terms of True Positives, False Positives, and False Negatives for both Components and Connections

Model–Prompt	Metric	Online Boutique Comp.	Online Boutique Conn.	Meetings System Comp.	Meetings System Conn.
	TP	11	6	7	6
o3–Zero-Shot	FN	1	12	6	11
	FP	11	20	4	17
	TP	11	7	6	8
o3–In-Context	FN	1	11	7	9
	FP	9	19	4	10
	TP	11	12	7	7
o3–Chain-of-Thought	FN	1	6	6	10
	FP	8	10	5	13
	TP	11	10	6	5
Gemini–Zero-Shot	FN	1	8	7	12
	FP	4	15	3	10
	TP	8	6	8	7
Gemini–In-Context	FN	4	12	5	10
	FP	3	11	2	4
	TP	10	5	8	7
Gemini–Chain-of-Thought	FN	2	13	5	10
	FP	16	16	4	7
	TP	9	9	4	0
Llama–Zero-Shot	FN	3	9	9	17
	FP	0	2	1	7
	TP	7	7	4	0
Llama–In-Context	FN	3	11	9	17
	FP	2	6	1	7
	TP	7	1	4	0
Llama–Chain-of-Thought	FN	5	17	9	17
	FP	4	11	1	7

to maintain uniformity in response length. The input length came at most 500 tokens including the system description and the prompts. These modest input/output tokens kept API costs low and enabled exhaustive experimentation across settings. Each model was evaluated under all three prompting techniques with prompts outlined in Figure 3, yielding nine model–prompt combinations. All our data and code are available in our GitHub repository for reproducibility.

V. Results

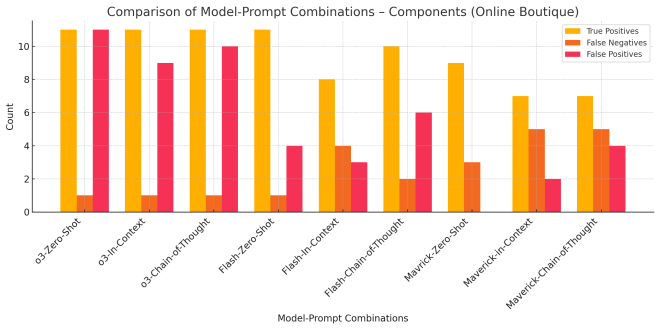
A. Single-Pass Generation

Table I shows the performance of the models in terms of TP, FP, FN on both components and connections, also visualized by Figures 4 and 5, while Table II shows the effectiveness of each model in terms of precision, recall, F1 score and edit distance for the two subject systems. Figures 6 and 7 show the edit distance for the two subject systems.

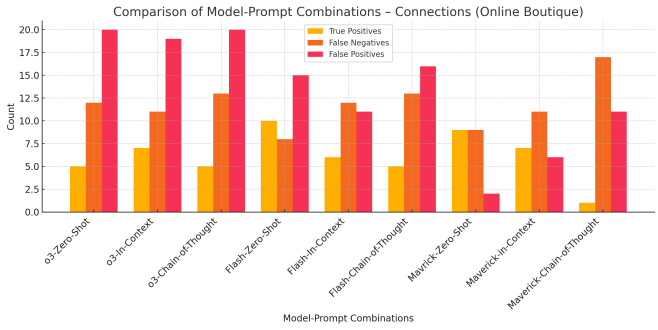
B. Results on the Online Boutique

Both Gemini and o3 achieve high TP counts and very low FN counts for components, indicating that they recover nearly all domain concepts. Llama, by contrast, records noticeably fewer TPs and more FNs, missing many domain concepts , so its representation of the domain is incomplete. Overall, the models perform well in generating domain concepts leading to a high recall.

o3 consistently produces the highest FP. A manual review shows these "extras" are not random noise or hallucination by the LLM but plausible extensions that guide the architect toward implementation — e.g., explicit *Payment-Gateway*, *Shipping-Carrier-Integration*, and *Recommendation-Engine* components. Gemini and Llama stick more strictly

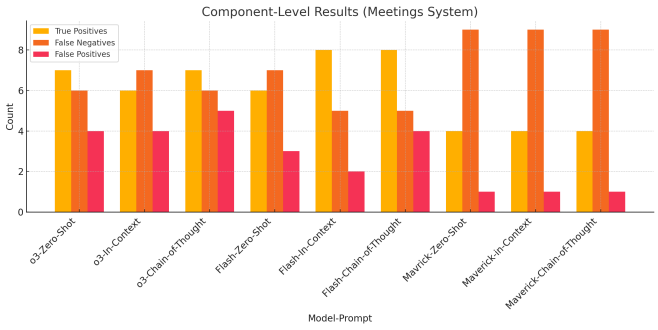


(a) Components

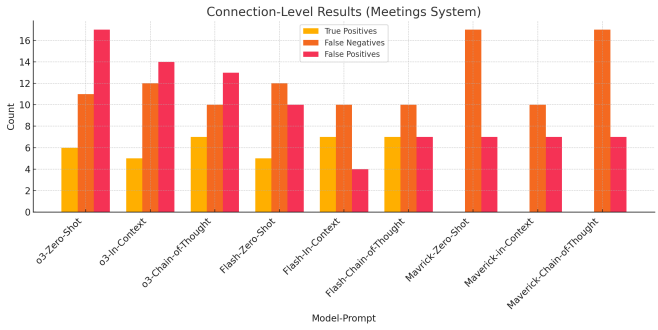


(b) Connections

Fig. 4: Results for *Online Boutique*: (a) component-level comparison and (b) connection-level comparison across model–prompt combinations.



(a) Components



(b) Connections

Fig. 5: Results for *Meetings System*: (a) component-level comparison and (b) connection-level comparison across model–prompt combinations.

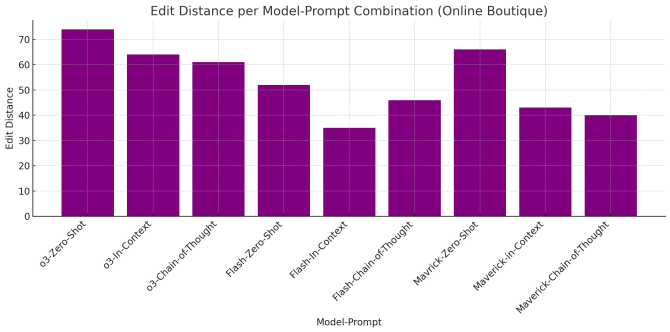


Fig. 6: Edit Distance for Online Boutique

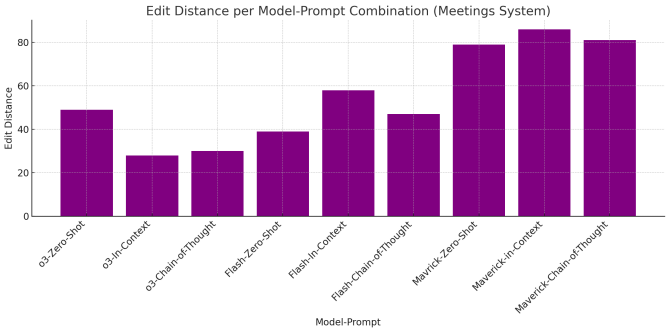


Fig. 7: Edit Distance for Meetings System

to the concepts named in the description and generate fewer additional components and have a higher precision.

o3's diagrams also register the largest edit distances across all models. The main driver is its systematic introduction of an API Gateway pattern in front of the domain services. This is a design construct, which is absent from the benchmark but a common architectural style in modern distributed systems. Gemini shows the smallest edit distance, especially under in-context prompting, where it captures nearly every domain concept while avoiding the API Gateway pattern, and hence is the closest to the benchmark architecture. Under zero-shot and chain-of-thought prompts Gemini adds an API gateway, producing a moderate distance with considerably fewer false positives. Llama's edit distance sits between Gemini and o3; it rarely proposes higher-level patterns, so it avoids large structural deviations, while it omits some domain entities.

Overall, all three models grasp the e-commerce domain, but Gemini offers the closest match to the hand-crafted architec-

TABLE II: Evaluation Scores per Model-Prompt Combination for Single Pass

Model	Metric	Online Boutique	Meetings Sys
o3-Zero-Shot	Edit Distance	74.00	49.00
	Precision	34.0%	38.2%
	Recall	55.2%	43.3%
	F1 Score	42.1%	40.6%
o3-In-Context	Edit Distance	64.00	28.00
	Precision	40.0%	54.8%
	Recall	60.0%	56.7%
	F1 Score	48.0%	55.7%
o3-Chain-of-Thought	Edit Distance	61.00	30.00
	Precision	34.8%	43.8%
	Recall	53.3%	46.7%
	F1 Score	42.1%	45.2%
Gemini-Zero-Shot	Edit Distance	52.00	39.00
	Precision	52.5%	45.8%
	Recall	70.0%	36.7%
	F1 Score	60.0%	40.7%
Gemini-In-Context	Edit Distance	35.00	58.00
	Precision	50.0%	71.4%
	Recall	46.7%	50.0%
	F1 Score	48.3%	58.8%
Gemini-Chain-of-Thought	Edit Distance	46.00	47.00
	Precision	40.5%	57.7%
	Recall	50.0%	50.0%
	F1 Score	44.8%	53.6%
Llama-Zero-Shot	Edit Distance	66.00	79.00
	Precision	90.0%	33.3%
	Recall	60.0%	13.3%
	F1 Score	72.0%	19.0%
Llama-In-Context	Edit Distance	43.00	86.00
	Precision	63.6%	33.3%
	Recall	46.7%	13.3%
	F1 Score	53.8%	19.0%
Llama-Chain-of-Thought	Edit Distance	40.00	81.00
	Precision	34.8%	33.3%
	Recall	26.7%	13.3%
	F1 Score	30.2%	19.0%

ture, o3 provides the most forward-leaning enhancements for both components and architectural style, and Llama delivers a minimal baseline that risks omitting critical capabilities.

C. Results on the Meetings System

Both o3 and Gemini again lead with high TP and low FN values, though they perform slightly worse than on Online Boutique. Manual inspection reveals one cause: the benchmark architecture assigns a separate database per module (because services communicate only through events). Only Gemini under chain-of-thought and *in-context* prompts replicates this pattern, which explains its relatively higher TP and lower FN.

o3 continues to introduce the most FPs, yet its additions—extra *Payment-Gateway*, *Notification-Service*, etc.—remain relevant to the domain. Gemini and Llama largely refrain from adding components, keeping FP counts minimal. Across all prompting strategies, Llama misses several key domain concepts (e.g., per-meeting fee processing and approval workflows), yielding t he lowest TP and highest FN totals.

The benchmark employs a single API entry point similar to an API gateway and an event-driven architecture where modules talk asynchronously via an event bus. Both o3 and Gemini reproduce the gateway pattern, so their edit distances are lower here than in Online Boutique. Yet only o3 with chain-of-thought prompting also adopts the event-bus style, resulting it the smallest overall distance and the closest structural match to the benchmark. Gemini's diagrams ignore the event bus so even with more true positives and creating separate databases for modules their edit distance is higher than o3 architectures. Llama proposes no higher-level patterns at all, leaving it with the highest edit distance of the group.

Taken together, these observations show that o3 and Gemini can generalize the API gateway pattern across domains, while o3 Chain of Thought is uniquely capable of recognizing and applying the underlying event-driven style. Llama trails in both domain coverage and architectural sophistication.

D. Two-Pass Iterative Refinement

Once again, we measured Precision, Recall and F1 score for components and connections combined and calculated a tree-based edit distance between the LLM diagram and the benchmark diagram for both the subject system. Results are shown in Table III. Figure 8 and Figure 9 show the heatmaps of precision for both subject systems.

Regarding the meetings system, the largest edit distance was 92 for the pair (o3-zero-shot, o3-zero-shot) and the smallest 12 for the pairs (Gemini-zero-shot, Llama-zero-shot) and (Gemini-zero-shot, Llama-in-context). The best choice for the first model in terms of top scores is the Gemini family, particularly Gemini-zero-shot and Gemini-in-context, with several entries achieving an edit distance no greater than 20. However, compared to o3 and Llama though, this group contained more syntactically invalid outputs, indicated by blank cells in the heatmaps. Concerning the second model, there seems to be no clearly better model, and the performance of the three model categories (i.e., o3, Llama, and Gemini) as well as the different prompt types (i.e., zero shot, in context, and CoT) is mixed. It is, however, noteworthy that Gemini-in-context as the second model produced three invalid outputs, making it an unreliable solution.

In addition to these scores themselves, their *variance* should also be taken into consideration. Particularly, the spread of the edit distances achieved by Gemini as either the first or the second model, measured by the different between the first and third quintiles, is smaller than that of Llama and o3. This indicates that, not only is Gemini better on average, but also that its performance is unlikely to deviate substantially from this mean. The other two models for their part witness greater performance variance, rendering them somewhat less trustworthy.

While the edit distance serves as a useful measure of overall performance, it fails to capture all the nuances of model behavior; specifically, trade-offs the model makes between false positives and false negatives. In this case, for the choice of the first model, Llama's recall on average is around 80%, indicating a failure to identify actual positive cases (that is, missing components and connections), whereas that of o3 and Gemini is better. Conversely, o3 has a poor *precision*, meaning it has a high false positive count (that is, redundant components and connections), but Gemini and Llama (especially the latter) achieve superior scores. Just as it was with the edit distance, there are no apparent trends among the performance

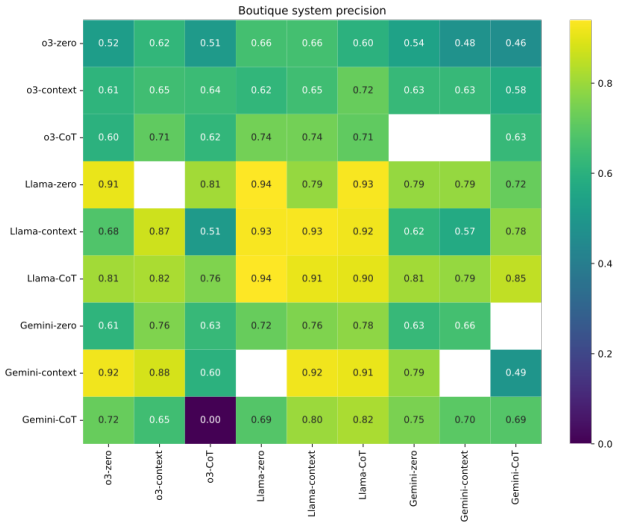


Fig. 8: Precision Heatmap for Boutique System

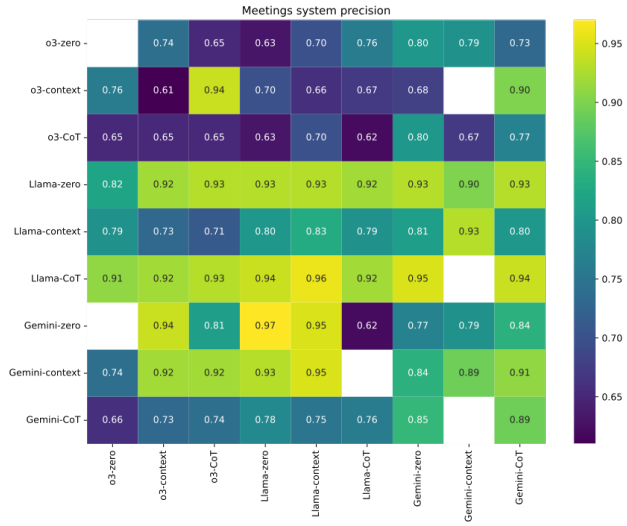


Fig. 9: Precision Heatmap for Meetings System

of the second models, and the selection of the first model is far more correlated with the final scores.

As for the other system, the online boutique, the variance of the edit distance was much greater, with the largest being 135 for the pair (o3-zero-shot, Gemini-CoT) and the smallest 13 for the pair (Gemini-in-context, Llama-CoT). For the choice of the first model, Llama achieves the best scores, followed by Gemini and then o3 in a distant third. When it comes to the choice of the second model, Gemini's performance lags behind that of o3 and Llama. Furthermore, Gemini as the second model undesirably leads to several invalid entries, as was the case with the meetings system.

The best recall is achieved by o3 as the first model, although its results are tainted by two invalid outputs. Gemini's recall score, though not on par with o3's, is still satisfactory, but Llama falls behind. o3 and Gemini also have the advantage of a smaller variance in terms of recall. For the choice of the second model, o3 remains at the top in terms of recall, with a practically negligible variance, but Llama is now superior to Gemini. o3's precision score is, however, lower, while Gemini's and Llama's are much better. Moreover, all three models suffer from a noticeable performance variance.

VI. Discussion and Limitations

A. Can LLMs generate Software Architecture

Across both case studies, o3 and Gemini were markedly better at recognizing and applying architectural styles than Llama. Taken together, the two models embedded an API Gateway pattern in 10 of the 12 model-prompt diagrams we examined, whereas Llama never proposed any overarching style. API Gateway is an important style often used in distributed systems.

Our best performing model, o3, under in-context and, especially, chain-of-thought prompting, delivers the *lowest* edit-distance values for one of the systems reproducing the benchmark's gateway pattern and its event-bus for asynchronous communication. Even for the other system, o3 earned a large edit-distance score not because it misunderstood the system requirements but because it *upgraded* the design from the benchmark. o3 remained the only system to generate an event driven architecture. This result also underlines how prompting strategy directly influences generated architectures quality. Consequently, LLMs are most effective when guided by an architect who can spell out the high-level steps of architectural synthesis, evaluate the model's suggestions, and decide whether to keep or discard the patterns it proposes.

When o3 was provided with more context, it started adhering more to the provided descriptions and improves its accuracy. Conversely, when Gemini and Llama were provided with more specific context, their performance was slightly reduced, especially for simpler systems, or remained largely the same. This shows that these models can be prone to "overthinking" [21] when presented with a simple task and asked to provide detailed reasoning. The effect is less prominent on more complex system, where additional context seemed to improve performance. Extra components proposed by o3 and some by Gemini were almost always relevant, serving as useful implementation pointers rather than random noise. Still, every model occasionally missed requirements or over-specified details, underscoring the need for informed human oversight.

For iterative refinement, after a thorough evaluation of all possible model configurations, we discovered this approach can bring about marked performance improvements, though it can also hurt performance or even generate invalid outputs if used unwisely. One of our chief findings was that the first model has a greater impact on performance than the second model, with Gemini generally being the most effective choice for the first model. However, as both the first and second model, Gemini has a tendency to produce invalid outputs, although compared to the total number of entries the fraction

TABLE III: Mean, First Quantile, and Third Quantile of Evaluation Metrics for Two-Pass Approach on the Online Boutique and Meetings Systems

Model	Metric	Boutique: As 1st			Boutique: As 2nd			Meetings: As 1st			Meetings: As 2nd		
		Mean	Q1	Q3	Mean	Q1	Q3	Mean	Q1	Q3	Mean	Q1	Q3
o3-Zero-Shot	Edit Distance	99	79	119	62	31	83	48	40	52	55	42	67
	Precision	56.3%	51.1%	62.0%	71.9%	61.5%	81.2%	73.2%	69.4%	77.1%	76.0%	70.3%	81.4%
	Recall	87.2%	82.6%	91.3%	89.1%	89.7%	91.0%	90.4%	90.2%	91.0%	83.1%	83.3%	91.2%
	F1 Score	68.0%	65.8%	73.1%	78.4%	73.9%	84.3%	80.1%	80.0%	82.0%	78.0%	77.6%	82.3%
o3-In-Context	Edit Distance	73	69	79	47	27	69	49	34	64	46	39	54
	Precision	64.4%	62.1%	65.2%	75.1%	65.3%	83.7%	74.0%	67.8%	80.3%	80.1%	73.7%	92.4%
	Recall	91.4%	91.3%	91.1%	90.2%	91.4%	92.0%	89.3%	88.4%	90.1%	79.2%	63.1%	90.3%
	F1 Score	75.3%	74.1%	76.2%	81.4%	76.5%	85.1%	81.0%	77.9%	84.0%	79.4%	74.8%	81.0%
o3-Chain-of-Thought	Edit Distance	63	49	79	78	64	91	57	53	68	36	26	54
	Precision	68.2%	63.0%	73.5%	56.7%	51.6%	64.9%	68.5%	65.3%	70.8%	81.4%	71.0%	93.2%
	Recall	92.1%	92.3%	92.0%	80.6%	90.1%	92.3%	91.2%	90.5%	91.1%	86.3%	81.0%	91.4%
	F1 Score	78.5%	74.2%	81.0%	66.4%	66.1%	75.0%	79.2%	76.6%	80.4%	84.0%	82.3%	86.2%
Gemini-Zero-Shot	Edit Distance	56	40	68	58	39	69	30	15	35	42	31	47
	Precision	69.1%	63.9%	76.3%	70.2%	63.4%	79.0%	84.3%	79.4%	94.1%	83.0%	80.5%	85.2%
	Recall	88.4%	88.0%	92.1%	77.8%	76.1%	79.4%	90.0%	88.2%	91.0%	80.4%	70.1%	90.4%
	F1 Score	78.3%	73.7%	83.0%	73.4%	70.8%	76.0%	86.1%	83.2%	92.0%	79.2%	77.4%	85.3%
Gemini-In-Context	Edit Distance	42	20	53	67	45	81	28	20	32	41	33	55
	Precision	79.4%	70.8%	92.0%	66.8%	60.4%	75.3%	89.3%	88.4%	92.2%	83.3%	79.1%	90.3%
	Recall	83.2%	78.7%	88.5%	78.4%	74.1%	79.3%	85.1%	81.6%	90.4%	80.3%	74.0%	89.1%
	F1 Score	80.1%	73.1%	89.0%	71.7%	69.2%	75.4%	86.0%	83.7%	89.2%	80.2%	77.8%	83.3%
Gemini-Chain-of-Thought	Edit Distance	47	36	56	73	42	96	41	34	45	41	27	50
	Precision	65.0%	69.1%	75.3%	65.6%	56.2%	74.2%	77.0%	74.1%	80.1%	86.4%	80.2%	91.0%
	Recall	78.5%	79.3%	92.2%	84.0%	78.1%	91.0%	90.4%	91.0%	92.2%	75.2%	63.0%	90.0%
	F1 Score	71.6%	76.2%	81.0%	73.2%	67.3%	77.3%	83.2%	82.1%	84.3%	79.3%	75.4%	85.2%
Llama-Zero-Shot	Edit Distance	43	39	49	48	39	58	47	43	57	47	36	67
	Precision	84.3%	79.2%	92.1%	78.4%	68.7%	93.0%	91.1%	92.4%	93.0%	81.3%	70.4%	93.2%
	Recall	72.7%	65.2%	78.4%	87.3%	87.8%	92.0%	67.4%	58.6%	69.1%	79.1%	69.3%	91.4%
	F1 Score	77.6%	75.1%	80.3%	81.3%	78.2%	84.1%	76.2%	71.7%	79.3%	79.1%	75.6%	84.0%
Llama-In-Context	Edit Distance	54	24	67	44	32	47	55	43	65	45	28	65
	Precision	76.0%	62.7%	92.0%	80.2%	74.4%	91.0%	80.0%	79.2%	81.1%	83.2%	70.4%	95.1%
	Recall	83.1%	78.1%	92.1%	84.2%	78.3%	92.0%	59.3%	52.4%	58.6%	78.2%	65.3%	91.3%
	F1 Score	78.4%	69.0%	87.4%	81.3%	77.7%	84.3%	68.1%	64.7%	73.4%	77.2%	70.5%	82.0%
Llama-Chain-of-Thought	Edit Distance	31	27	40	41	27	52	51	43	57	58	48	66
	Precision	84.2%	81.0%	90.0%	81.3%	72.8%	91.0%	93.3%	92.0%	94.2%	76.1%	66.4%	82.0%
	Recall	82.1%	78.6%	89.0%	86.3%	83.1%	92.1%	63.2%	59.7%	68.1%	81.0%	63.2%	92.3%
	F1 Score	83.2%	81.4%	85.3%	83.4%	80.1%	87.2%	74.1%	71.3%	79.1%	76.2%	74.5%	82.3%

of invalid ones is negligible.

More specifically, o3's recall is superior, whereas it is outperformed by Gemini and Llama in terms of precision. This signifies that o3's outputs are comprehensive and successfully take into account important components and connections, which are often overlooked by the other two models. On the other hand, its higher precision translates into more false positives, namely, superfluous components or connections not present in the ground truth. A particularly powerful combination is Llama as the first model and o3 as the second one, which maintains an excellent balance between precision, recall, and overall distance. Besides performance metrics, another important consideration is performance *variance*, that is, the spread of the edit distance, precision, recall, or F1 score when using a particular LLM as the first or second model. Even if an LLM's performance is on average superior, a high variance in its scores indicates it cannot consistently attain adequate results and instead sometimes performs very well, at other times quite poorly. We found that models with a better average performance also tend to have a smaller variance, suggesting a correlation between robustness and effectiveness.

To conclude, current LLMs are best viewed as capable assistants that can draft an initial component map, recall common patterns, and suggest pragmatic extensions, but they still rely on human architects to validate coverage, prune irrelevancies, and steer the design toward the desired architectural style.

B. Limitations

Our study's primary limitation is its ability to assess generalization. Because the benchmark systems were drawn from public GitHub projects created in 2019–2020 (and subsequently updated), portions of their descriptions or diagrams may have appeared in the models' pre-training corpora. This potential data leakage weakens external validity: strong results could partly reflect training familiarity rather than true out-of-distribution competence. Compounding this, our dataset is small (two systems), which further limits what we can infer about model behavior across diverse domains, scales, and architectural styles.

Addressing these issues is a priority for future work. We plan to adopt time-split evaluations (only using repositories created after a model's cutoff date), incorporate private or industrial datasets that are absent from public training corpora, and broaden coverage beyond microservice-centric systems to include less common or emerging architectures (e.g., event-sourced monoliths, cyber-physical systems, high-throughput data pipelines). We will also expand the sample size to better probe robustness, variance across runs, and sensitivity to prompt design.

That said, the potential overlap between training data and architects queries also reflects typical usage scenarios where architects often seek guidance from open-source examples. In contexts where the target system resembles publicly doc-

umented projects, a certain degree of "training familiarity" is unavoidable, and arguably desirable, because it allows the LLM to reuse proven patterns. Our results therefore remain informative for use cases in which the model is asked to design a system that lies within the distribution of its training corpus.

Finally, the real-world impact of our approach remains to be demonstrated. It is still unclear how effectively the proposed tool integrates into existing architecture workflows, whether it meaningfully reduces time spent on early design, and how feasible it is for larger projects with many teams, bounded contexts, and evolving requirements. Future studies should examine practitioner adoption, cost and latency trade-offs, collaboration patterns (human-in-the-loop refinement), and longitudinal outcomes—e.g., whether LLM-assisted architectures lead to fewer redesigns, clearer interfaces, or improved change resilience over time.

VII. Conclusion

In this study, we set out to determine if Large Language Models can be used to generate high-level software architecture diagrams from purely textual descriptions. Using OpenAI o3, Gemini 2.0 Flash and Llama 4.0 Maverick with various prompt techniques, including zero-shot, in-context and chain-of-thought, we observed that the strongest models, o3 and Gemini, consistently recovered most domain concepts and the relationships between them in both in the two systems we studied belonging to different domains, whereas Llama lagged behind, especially on the second, less familiar domain. Results significantly improved when two models were combined, where one performed a first pass of the requirements and the second was used as refinement. When the models were provided with more context and were asked to provide more in-depth reasoning, their performance was improved but only for more complex systems.

While the results are encouraging, more research is needed to exploit the full potential of generative AI for software architecture. More specifically, our study focuses mainly on off-the-shelf use of LLMs without any additional find-tuning or the use of RAG (retrieval-augmented generation). Our study also employs one-shot prompting, but no interactive methods were tested. In addition, our study focuses primarily on recovering the structural properties of the architecture without particular emphasis on the functional and non-functional properties. Finally, while it would be useful to compare the performance of LLMs against existing software architecture recovery methods, it is important to note that in our study, the LLMs only relied on documentation and textual requirements with no access to code.

References

[1] M. Shaw and D. Garlan, *Software architecture: perspectives on an emerging discipline*. USA: Prentice-Hall, Inc., 1996.

[2] C. Hofmeister, P. Kruchten, R. L. Nord, H. Obbink, A. Ran, and P. America, "A general model of software architecture design derived from five industrial approaches," *Journal of Systems and Software*, vol. 80, no. 1, pp. 106–126, 2007. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121206001634

[3] OpenAI. (2023) ChatGPT: Optimizing Language Models for Dialogue. [Online]. Available: https://www.openai.com/chatgpt

[4] E. Souza, A. Moreira, and M. Goulão, "Deriving architectural models from requirements specifications: a systematic mapping study," *Information and Software Technology*, vol. 109, 05 2019.

[5] A. S. Pothukuchi, L. V. Kota, and V. Mallikarjunaradhya, "Impact of generative ai on the software development lifecycle (sdlc)," *International Journal of Creative Research Thoughts*, vol. 11, no. 8, 2023.

[6] Z. Zheng, K. Ning, Q. Zhong, J. Chen, W. Chen, L. Guo, W. Wang, and Y. Wang, "Towards an understanding of large language models in software engineering tasks," 2024. [Online]. Available: https://arxiv.org/abs/2308.11396

[7] A. Ferrari, S. Abualhaija, and C. Arora, "Model generation with llms: From requirements to uml sequence diagrams," in *2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW)*, 2024, pp. 291–300.

[8] J. Cámara, J. Troya, J. Montes-Torres, and F. J. Jaime, "Generative ai in the software modeling classroom: An experience report with chatgpt and unified modeling language," *IEEE Software*, vol. 41, no. 6, pp. 73–81, 2024.

[9] M. Krishna, B. Gaur, A. Verma, and P. Jalote, "Using llms in software requirements specifications: An empirical evaluation," in *2024 IEEE 32nd International Requirements Engineering Conference (RE)*. IEEE, Jun. 2024, p. 475–483. [Online]. Available: http://dx.doi.org/10.1109/RE59067.2024.00056

[10] R. Dhar, K. Vaidhyanathan, and V. Varma, "Can llms generate architectural design decisions? -an exploratory empirical study," 2024. [Online]. Available: https://arxiv.org/abs/2403.01709

[11] A. Ahmad, M. Waseem, P. Liang, M. Fahmideh, M. S. Aktar, and T. Mikkonen, "Towards human-bot collaborative software architecting with chatgpt," in *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 279–285. [Online]. Available: https://doi.org/10.1145/3593434.3593468

[12] T. Eisenreich, S. Speth, and S. Wagner, "From requirements to architecture: An ai-based journey to semi-automatically generate software architectures," in *Proceedings of the 1st International Workshop on Designing Software*, ser. Designing '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 52–55. [Online]. Available: https://doi.org/10.1145/3643660.3643942

[13] I. Malavolta, P. Lago, H. Muccini, P. Pelliccione, and A. Tang, "What industry needs from architectural languages: A survey," *IEEE Transactions on Software Engineering*, vol. 39, no. 6, pp. 869–891, 2013.

[14] Y. Li, "A practical survey on zero-shot prompt design for in-context learning," in *Proceedings of the Conference Recent Advances in Natural Language Processing - Large Language Models for Natural Language Processings*, ser. RANLP. INCOMA Ltd., Shoumen, BULGARIA, 2023, p. 641–647. [Online]. Available: {http://dx.doi.org/10.26615/978-954-452-092-2_069}

[15] Q. Dong, L. Li, D. Dai, C. Zheng, J. Ma, R. Li, H. Xia, J. Xu, Z. Wu, T. Liu *et al.*, "A survey on in-context learning," *arXiv preprint arXiv:2301.00234*, 2022.

[16] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," 2023. [Online]. Available: https://arxiv.org/abs/2201.11903

[17] P. Chen, Z. Guo, B. Haddow, and K. Heafield, "Iterative translation refinement with large language models," *arXiv preprint arXiv:2306.03856*, 2023.

[18] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in Neural Information Processing Systems*, vol. 36, pp. 46 534–46 594, 2023.

[19] C. Qian, J. Li, Y. Dang, W. Liu, Y. Wang, Z. Xie, W. Chen, C. Yang, Y. Zhang, Z. Liu *et al.*, "Iterative experience refinement of software-developing agents," *arXiv preprint arXiv:2405.04219*, 2024.

[20] B. Paaßen, "Revisiting the tree edit distance and its backtracing: A tutorial," 2022. [Online]. Available: https://arxiv.org/abs/1805.06869

[21] Y. Sui, Y.-N. Chuang, G. Wang, J. Zhang, T. Zhang, J. Yuan, H. Liu, A. Wen, S. Zhong, H. Chen *et al.*, "Stop overthinking: A survey on efficient reasoning for large language models," *arXiv preprint arXiv:2503.16419*, 2025.